Université Abdelmalek Essaâdi
École Nationale des Sciences Appliquées – Al Hoceima
TP – J2EE, Java EE (JEE) ou Jakarta EE
TDIA2-S4 / 2025-26
Pr. Mohamed CHERRADI

TP N° 5:

Réalisés par: Asmae Daouad.

Introduction et objectifs:

Ce travail pratique consiste à développer une application web Java EE basée sur l'architecture MVC2 (Model-View-Controller) et le modèle **3-tiers**, en utilisant **Hibernate comme framework ORM**.

L'application permet la gestion complète des produits avec authentification et gestion des rôles.

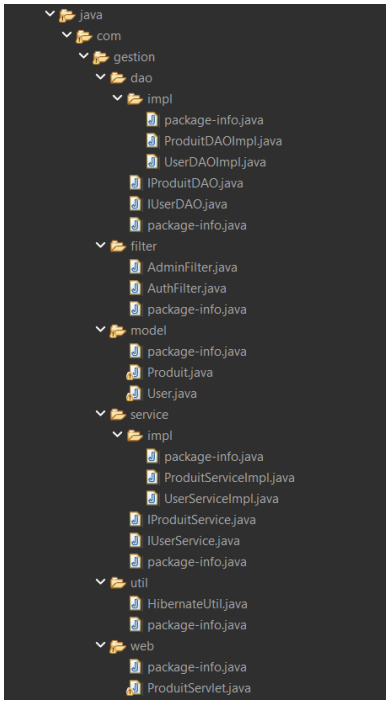
Objectifs du TP:

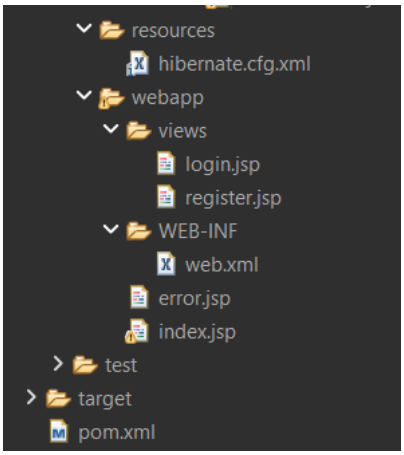
- Mettre en œuvre l'architecture **MVC2 (Front Controller avec Servlet centrale)**
- Implémenter un système d'authentification avec **sessions HTTP**
- Gérer les rôles utilisateur (**ADMIN / USER**) via des **Filters**
- Utiliser **JSTL** pour la couche vue (JSP)
- Utiliser **Hibernate** pour la persistance des données
- Mettre en place une architecture extensible avec :

  - DAO (Data Access Object)
  - Service (logique métier)

Architecture du projet:

Le projet suit une architecture 3-tiers combinée avec le patron MVC2 :





Couche Modèle (model/):

La couche Modèle représente les données manipulées par l'application.
Dans cette version basée sur **MVC2 avec Hibernate**, les modèles sont des **classes Java annotées (JPA Entities)**, contrairement au TP précédent où il s'agissait de simples POJO.

Ces classes sont utilisées pour :

- Représenter les tables de la base de données
- Transporter les données entre les couches (DAO, Service, Controller)
- Permettre la persistance automatique grâce à Hibernate

User.java:

```java
package com.gestion.model;

import jakarta.persistence.*;
import java.io.Serializable;

@Entity
@Table(name = "users")
public class User implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false, length = 50)
    private String username;

    @Column(nullable = false, length = 255)
    private String password;

    @Column(nullable = false, length = 100)
    private String email;

    @Column(length = 20)
    private String phone;

    @Column(nullable = false, length = 10)
    private String role;

    public User() {}

    public User(String username, String password, String email, String phone, String role) {
        this.username = username;
        this.password = password;
        this.email = email;
        this.phone = phone;
        this.role = role;
    }
```

```java
// Getters et Setters
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getUsername() {
    return username;
}

public void setUsername(String username) {
    this.username = username;
}

public String getPassword() {
    return password;
}

public void setPassword(String password) {
    this.password = password;
}

public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getPhone() {
    return phone;
}
```

Produit.java:

```java
package com.gestion.model;

import jakarta.persistence.*;
import java.io.Serializable;

@Entity
@Table(name = "produits")
public class Produit implements Serializable {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nom;

    @Column(length = 500)
    private String description;

    @Column(nullable = false)
    private Double prix;

    public Produit() {}

    public Produit(String nom, String description, Double prix) {
        this.nom = nom;
        this.description = description;
        this.prix = prix;
    }

    // Getters et Setters
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
```

```java
public void setId(Long id) {
    this.id = id;
}

public String getNom() {
    return nom;
}

public void setNom(String nom) {
    this.nom = nom;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}

public Double getPrix() {
    return prix;
}

public void setPrix(Double prix) {
    this.prix = prix;
}
```

Couche DAO (dao/ et dao/impl/):

- L'interface IProduitDAO joue un rôle fondamental dans l'architecture de l'application en respectant le principe **Open/Closed** :
- Une classe doit être **ouverte à l'extension** mais **fermée à la modification**.
- Elle permet de définir un **contrat** pour les opérations de persistance sans dépendre d'une implémentation spécifique.

IProduitDAO.java – Interface :

```java
package com.gestion.dao;

import com.gestion.model.Produit;
import java.util.List;

public interface IProduitDAO {
    void add(Produit produit);
    void delete(Long id);
    void update(Produit produit);
    Produit findById(Long id);
    List<Produit> findAll();
}
```

ProduitDAOImpl.java – Implémentation(Map en mémoire):

```java
package com.gestion.dao.impl;

import com.gestion.dao.IProduitDAO;
import com.gestion.model.Produit;
import com.gestion.util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;
import org.hibernate.query.Query;
import java.util.List;

public class ProduitDAOImpl implements IProduitDAO {



    public ProduitDAOImpl() {
    }

    @Override
    public void add(Produit produit) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            session.persist(produit);
            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) transaction.rollback();
            e.printStackTrace();
        }
    }

    @Override
    public void delete(Long id) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            Produit produit = session.get(Produit.class, id);
            if (produit != null) session.remove(produit);
```

```java
            Produit produit = session.get(Produit.class, id);
            if (produit != null) session.remove(produit);
            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) transaction.rollback();
            e.printStackTrace();
        }
    }

    @Override
    public void update(Produit produit) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            session.merge(produit);
            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) transaction.rollback();
            e.printStackTrace();
        }
    }

    @Override
    public Produit findById(Long id) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            return session.get(Produit.class, id);
        }
    }

    @Override
    public List<Produit> findAll() {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            Query<Produit> query = session.createQuery("FROM Produit ORDER BY id", Produit.class);
            return query.list();
        }
    }
}
```

Couche Service (service/ et service/impl/):

La couche Service contient la logique métier. Elle est appelée par les Servlets et appelle le DAO.

```java
package com.gestion.service;

import com.gestion.model.Produit;
import java.util.List;

public interface IProduitService {
    void addProduit(Produit produit);
    void deleteProduit(Long id);
    void updateProduit(Produit produit);
    Produit getProduitById(Long id);
    List<Produit> getAllProduits();
}
```

ProduitServiceImpl.java – Exemple de logique métier:

```java
import com.gestion.dao.IProduitDAO;
import com.gestion.dao.impl.ProduitDAOImpl;
import com.gestion.model.Produit;
import com.gestion.service.IProduitService;
import java.util.List;

public class ProduitServiceImpl implements IProduitService {

    private IProduitDAO produitDAO;

    public ProduitServiceImpl() {
        this.produitDAO = new ProduitDAOImpl();  // Nouvelle instance
    }

    @Override
    public void addProduit(Produit produit) {
        produitDAO.add(produit);
    }

    @Override
    public void deleteProduit(Long id) {
        produitDAO.delete(id);
    }

    @Override
    public void updateProduit(Produit produit) {
        produitDAO.update(produit);
    }

    @Override
    public Produit getProduitById(Long id) {
        return produitDAO.findById(id);
    }

    @Override
    public List<Produit> getAllProduits() {
        return produitDAO.findAll();
```

Filtres HTTP (filter/):

Les filtres interceptent les requêtes AVANT qu'elles atteignent les Servlets. Ils permettent d'appliquer des contrôles de sécurité de manière centralisée.

AuthFilter.java – Vérification de la session:

```java
package com.gestion.filter;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest req, ServletResponse resp, FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) resp;

        String action = request.getParameter("action");


        if ("login".equals(action) || "register".equals(action)) {
            chain.doFilter(req, resp);
            return;
        }

        HttpSession session = request.getSession(false);
        boolean connected = session != null && session.getAttribute("user") != null;

        if (connected) {
            chain.doFilter(req, resp);
        } else {
            response.sendRedirect(request.getContextPath() + "/produit?action=login");
        }
    }
}
```

AdminFilter.java – Vérification du role:

```java
package com.gestion.filter;

import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.IOException;

public class AdminFilter implements Filter {

    @Override
    public void doFilter(ServletRequest req, ServletResponse resp, FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest request = (HttpServletRequest) req;
        HttpServletResponse response = (HttpServletResponse) resp;

        String action = request.getParameter("action");


        boolean needsAdmin = "add".equals(action) || "delete".equals(action) ||
                             "edit".equals(action) || "update".equals(action);

        if (!needsAdmin) {
            chain.doFilter(req, resp);
            return;
        }

        HttpSession session = request.getSession(false);
        String role = (session != null) ? (String) session.getAttribute("role") : null;

        if ("ADMIN".equals(role)) {
            chain.doFilter(req, resp);
        } else {
            response.sendRedirect(request.getContextPath() + "/produit?erreur=acces_refuse");
        }
    }
}
```

L'ordre est garanti par la configuration web.xml.

Flux : Requête --> AuthFilter --> AdminFilter --> Servlet (chaque étape peut bloquer).

Configuration web.xml:

L'ordre des filtres est essentiel. Dans notre application, AuthFilter doit s'exécuter avant AdminFilter pour vérifier d'abord la connexion, puis les droits administrateur. Avec les annotations @WebFilter, l'ordre n'est pas garanti. Pour contrôler l'ordre, il faut déclarer les filtres dans web.xml dans l'ordre souhaité ou fusionner les deux filtres en un seul.

```
        xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee
        https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
        version="6.0">

    <display-name>Gestion Produits</display-name>

    <session-config>
        <session-timeout>30</session-timeout>
    </session-config>

    <welcome-file-list>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>

    <!-- Déclaration des filtres -->
    <filter>
        <filter-name>AuthFilter</filter-name>
        <filter-class>com.gestion.filter.AuthFilter</filter-class>
    </filter>

    <filter>
        <filter-name>AdminFilter</filter-name>
        <filter-class>com.gestion.filter.AdminFilter</filter-class>
    </filter>

    <!-- Mapping des filtres -->
    <filter-mapping>
        <filter-name>AuthFilter</filter-name>
        <url-pattern>/produit</url-pattern>
    </filter-mapping>

    <filter-mapping>
        <filter-name>AdminFilter</filter-name>
        <url-pattern>/produit</url-pattern>
    </filter-mapping>
</web-app>
```

Flux d'une requête HTTP (MVC2

```
Navigateur
  |
  |-- /produit?action=add
  v
AuthFilter → (non connecté → login)
  |
AdminFilter → (pas ADMIN → refus)
  |
ProduitServlet (Front Controller)
  |
ProduitServiceImpl
  |
ProduitDAOImpl (Hibernate)
  |
Base de données
  |
ProduitServlet
  |
  |-- redirect /produit?action=list
  v
JSP (index.jsp)
  |
HTML affiché
```

Couche Web – Servlets (web/):

Dans cette application basée sur **l'architecture MVC2**, la couche Web repose sur une **Servlet unique appelée** ProduitServlet, jouant le rôle de **Front Controller**.

Contrairement au TP précédent (MVC1) où chaque action était gérée par une servlet spécifique, ici **toutes les requêtes passent par une seule servlet**, ce qui centralise le contrôle et améliore la maintenabilité.

```java
@WebServlet("/produit")
public class ProduitServlet extends HttpServlet {

    private IProduitService produitService;
    private IUserService userService;

    @Override
    public void init() throws ServletException {

        produitService = new ProduitServiceImpl();
        userService = new UserServiceImpl();
    }

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        String action = req.getParameter("action");

        if (action == null) action = "list";

        switch (action) {
            case "login":
                showLogin(req, resp);
                break;
            case "register":
                showRegister(req, resp);
                break;
            case "logout":
                logout(req, resp);
                break;
            case "delete":
                deleteProduit(req, resp);
                break;
            case "edit":
                showEditForm(req, resp);
                break;
```

```java
// ========== PRODUITS METHODS ==========

private void listProduits(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
    String idParam = req.getParameter("idProduit");
    List<Produit> liste;

    if (idParam != null && !idParam.trim().isEmpty()) {
        try {
            Produit p = produitService.getProduitById(Long.parseLong(idParam));
            if (p != null) {
                liste = Arrays.asList(p);
            } else {
                liste = produitService.getAllProduits();
                req.setAttribute("error", "Aucun produit trouvé avec l'ID " + idParam);
            }
        } catch (NumberFormatException e) {
            liste = produitService.getAllProduits();
            req.setAttribute("error", "ID invalide.");
        }
    } else {
        liste = produitService.getAllProduits();
    }

    req.setAttribute("listeProduits", liste);
    req.getRequestDispatcher("/index.jsp").forward(req, resp);
}

private void showEditForm(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
    try {
        Long id = Long.parseLong(req.getParameter("id"));
        Produit p = produitService.getProduitById(id);
        if (p != null) {
            req.setAttribute("produitEdit", p);
        } else {
            req.setAttribute("error", "Produit introuvable.");
```

Vues JSP (webapp/views/ et index.jsp):

Les JSP utilisent exclusivement JSTL (Jakarta Standard Tag Library) pour la logique d'affichage. Aucun scriptlet Java (<% %>) n'est utilisé, conformément aux bonnes pratiques MVC.

index.jsp – Extrait clé : formulaire dual Ajout / Modification:

```jsp
<%@ page contentType="text/html;charset=UTF-8" language="java" %>
<%@ taglib uri="jakarta.tags.core" prefix="c" %>
<%
    // Vérification de session - FORCER LA CONNEXION
    if (session == null || session.getAttribute("user") == null) {
        response.sendRedirect(request.getContextPath() + "/produit?action=login");
        return;
    }
%>
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Gestion des Produits</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>

<div class="container mt-4">
    <div class="d-flex justify-content-between align-items-center border-bottom pb-2 mb-4">
        <h3>Gestion des Produits</h3>
        <div>
            <span>Bonjour, <strong>${sessionScope.user.username}</strong></span>
            <c:choose>
                <c:when test="${sessionScope.role == 'ADMIN'}">
                    <span class="text-danger">[ADMIN]</span>
                </c:when>
                <c:otherwise>
                    <span class="text-secondary">[USER]</span>
                </c:otherwise>
            </c:choose>
            <a href="${pageContext.request.contextPath}/produit?action=logout" class="btn btn-link btn-sm">Déconnexion</a>
        </div>
    </div>

    <c:if test="${param.erreur == 'acces_refuse'}">
        <div class="alert alert-danger">Accès refusé : réservé aux administrateurs.</div>
```

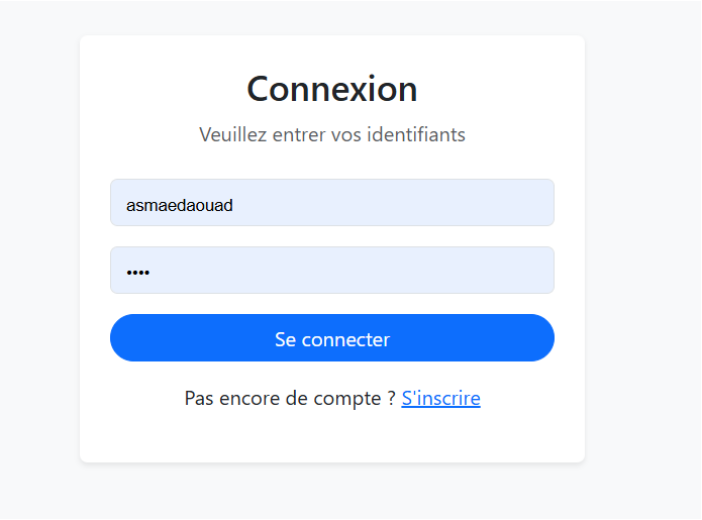
```jsp
    </form>
</div>

<!-- Tableau produits -->
<table class="table table-bordered">
    <thead class="table-light">
        <tr>
            <th>ID</th>
            <th>Nom</th>
            <th>Description</th>
            <th>Prix</th>
            <c:if test="${sessionScope.role == 'ADMIN'}">
                <th>Actions</th>
            </c:if>
        </tr>
    </thead>
    <tbody>
        <c:choose>
            <c:when test="${empty listeProduits}">
                <tr>
                    <td colspan="5" class="text-center">Aucun produit trouvé.</td>
                </tr>
            </c:when>
            <c:otherwise>
                <c:forEach var="p" items="${listeProduits}">
                    <tr>
                        <td>${p.id}</td>
                        <td><strong>${p.nom}</strong></td>
                        <td>${p.description}</td>
                        <td>${p.prix} DH</td>
                        <c:if test="${sessionScope.role == 'ADMIN'}">
                            <td>
                                <a href="${pageContext.request.contextPath}/produit?action=edit&id=${p.id}" class="btn btn-sm btn-outline-
                                <a href="${pageContext.request.contextPath}/produit?action=delete&id=${p.id}"
                                   class="btn btn-sm btn-outline-danger"
                                   onclick="return confirm('Supprimer ${p.nom} ?')">Supprimer</a>
                            </td>
                        </c:if>
```

1. L'utilisateur"Admin" clique sur **"Modifier"** → lien href='/editProduit?id=3'
2. **EditProduitServlet** récupère le produit correspondant et fait req.setAttribute("produitEdit", p)
3. La requête est **forward** vers index.jsp → le JSP détecte que produitEdit != null
4. Le formulaire est alors **pré-rempli** avec les valeurs du produit à modifier
5. L'action du formulaire devient updateProduit au lieu de addProduit

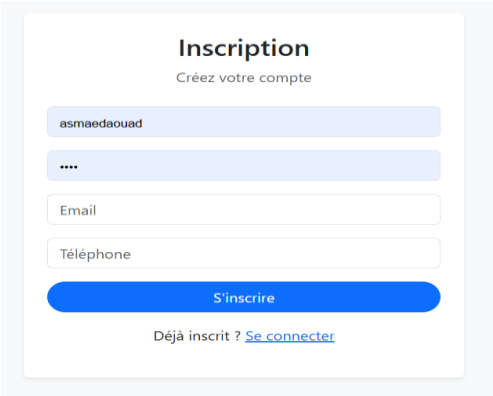
Interfaces de l'application:

Les captures d'écran ci-dessous illustrent les principales vues de l'application. Elles sont à insérer manuellement depuis votre navigateur.

Page de connexion (login.jsp):



Page d'inscription (register.jsp):



Page principale – Vue ADMIN (index.jsp):

Page principale – Vue USER (index.jsp) :

Gestion des Produits

Bonjour, **asmaedaouad** [USER] Déconnexion

Rechercher par ID | Rechercher | Réinitialiser

ID	Nom	Description	Prix
1	**Clavier Mécanique**	Cherry MX Red, rétro-éclairé RGB	299.0 DH
2	**Souris Gaming**	Sans fil 16000 DPI, ergonomique	149.0 DH
3	**Écran 27"**	4K IPS 144Hz HDR400	1299.0 DH
4	**Casque Audio**	Bluetooth 5.0, réduction de bruit	399.0 DH

Conclusion:

Ce travail pratique nous a permis de concevoir et développer une application web Java basée sur une architecture **MVC2** associée au modèle **3-tiers**, en utilisant **Maven** pour la gestion du projet et **Hibernate** comme framework de persistance.

L'adoption du modèle MVC2, avec une **Servlet Front Controller**, a permis de centraliser le traitement des requêtes et de mieux structurer l'application, contrairement à l'approche MVC1 utilisée précédemment. Cette organisation améliore la lisibilité, la maintenabilité et la réutilisabilité du code.

L'intégration de **Hibernate** a facilité la gestion de la persistance des données en éliminant l'utilisation directe du SQL et en assurant une correspondance automatique entre les objets Java et les tables de la base de données. Cela rend l'application plus flexible et moins dépendante du système de gestion de base de données.

L'utilisation de **Maven** a permis une gestion efficace des dépendances et une structuration standardisée du projet, facilitant ainsi son déploiement et son évolution.

Par ailleurs, la mise en place d'une architecture en couches (DAO, Service, Controller) ainsi que l'utilisation d'interfaces a permis de respecter les principes de conception tels que le **faible couplage**, la **séparation des responsabilités** et le principe **Open/Closed**.

Enfin, l'implémentation d'un système d'authentification avec gestion des rôles (ADMIN / USER) a permis de sécuriser l'accès aux différentes fonctionnalités de l'application.